

Regular Language

A **regular language** is a formal language that can be described using:

- **Regular expressions**, or
- **Finite automata** (Deterministic or Non-deterministic), or
- **Regular grammars**

Formal Definition:

A language L over an alphabet Σ is **regular** if there exists a **finite automaton** that accepts all strings in L and rejects all others.

Example:

- $L = \{w \in \{0,1\}^* \mid w \text{ ends with } 01\}$ is regular.
- It can be accepted by a DFA and represented by the regular expression $(0+1)^*01(0+1)^*01(0+1)^*01$.

Regular Expression

A **regular expression (RE)** is an algebraic notation used to describe a **regular language**. It uses the following operators:

- **Union (+ or |)**: $R+SR$ or $R|SR$ (either R or S)
- **Concatenation**: RS (R followed by S)
- **Kleene Star (*)**: R^* (zero or more repetitions of R)

Example:

- The RE $(a+b)^*ab(a+b)^*ab(a+b)^*ab$ represents all strings over $\{a,b\}$ that end with "**ab**".
- This language is regular.

Relation:

Every regular expression defines a **regular language**, and for every regular language, there exists an equivalent regular expression.

Comparison: Regular Language, Regular Expression, Finite Automata, and Regular Grammar

Aspect	Regular Language	Regular Expression	Finite Automata	Regular Grammar
Definition	A set of strings over an alphabet that can be recognized by a finite automaton.	An algebraic notation used to describe regular languages using operators like union, concatenation, and Kleene star.	A mathematical model of computation used to recognize regular languages.	A grammar where all production rules are of the form $A \rightarrow aB$, $A \rightarrow a$, or $A \rightarrow \epsilon$.
Purpose	Specifies which strings are part of the language.	Provides a symbolic way to describe the pattern of strings in a regular language.	Acts as a recognizer to accept or reject strings in a regular language.	Generates all strings in a regular language using production rules.
Example	$L = \{w \in \{0,1\}^* \mid w \text{ ends with } 01\}$ $L = \{ w \in \{0,1\}^* \mid w \text{ ends with } 01 \}$	$(0+1)^*01(0+1)^*01(0+1)^*01$	DFA/NFA accepting all strings ending in "01"	$S \rightarrow 0S \mid 1S \mid 0A; A \rightarrow 1$
Equivalence	Defined by a regular expression, accepted by finite automata, and generated by a regular grammar.	Can be converted to an equivalent finite automaton.	Can be converted to an equivalent regular expression or regular grammar.	Can be converted to an equivalent finite automaton or regular expression.

Algebraic Laws of Regular Expressions – With Examples

Regular expressions follow algebraic laws that allow simplification, transformation, and checking of equivalence between expressions. Let R, S, T be regular expressions over an alphabet Σ , $\varnothing$ denote the empty set, and ϵ denote the language containing the empty string.

Law Name	Law Statement	Example
Identity Laws	$R + \varnothing = R$ $R\epsilon = \epsilon R = R$	$(0+1) + \varnothing = (0+1)$ $(ab)\epsilon = ab$
Null Laws	$R\varnothing = \varnothing R = \varnothing$	$(0+1)\varnothing = \varnothing$

Idempotent Law	$R + R = R$	$(ab) + (ab) = ab$
Commutative Laws	$R + S = S + R$	$0+1 = 1+0$
Associative Laws	$R + (S + T) = (R + S) + T$ $R(ST) = (RS)T$	$(0+(1+2)) = ((0+1)+2)$
Distributive Laws	$R(S + T) = RS + RT$ $(R + S)T = RT + ST$	$a(b+c) = ab + ac$
Closure Laws	$(R^*)^* = R^*$ $R^* = \varepsilon + RR^*$	$(a^*)^* = a^*$
Annihilation Laws	$\varphi^* = \varepsilon$	Kleene star of φ results in ε
Absorption Laws	$R + RR^* = R^*$	Since R^* already contains R